

OrderDAOImpl.java

```
package com.harisudhan.harisudhanmart.dao;

import com.harisudhan.harisudhanmart.model.Order;
import com.harisudhan.harisudhanmart.model.OrderItem;
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;
import java.sql.Timestamp;
import java.util.ArrayList;
import java.util.List;
import java.util.Optional;
import javax.sql.DataSource;

public class OrderDAOImpl implements OrderDAO {

    private final DataSource dataSource;

    public OrderDAOImpl(DataSource dataSource) {
        this.dataSource = dataSource;
    }

    /**
     * Inserts the order and its line items using the caller's transactional Connection.
     * OrderService owns commit/rollback so the whole checkout is atomic.
     */
    @Override
    public Order createOrder(Connection conn, Order order, List<OrderItem> items) throws SQLException {
        String orderSql = "INSERT INTO orders (buyer_id, status, total_amount) VALUES (?, ?, ?)";
        try (PreparedStatement ps = conn.prepareStatement(orderSql, Statement.RETURN_GENERATED_KEYS)) {
            ps.setLong(1, order.getBuyerId());
            ps.setString(2, order.getStatus().name());
            ps.setBigDecimal(3, order.getTotalAmount());
            ps.executeUpdate();
            try (ResultSet keys = ps.getGeneratedKeys()) {
                if (keys.next()) {
                    order.setId(keys.getLong(1));
                }
            }
        }

        String itemSql = "INSERT INTO order_items (order_id, product_id, quantity, unit_price) VALUES (?, ?, ?, ?)";
        try (PreparedStatement ps = conn.prepareStatement(itemSql)) {
            for (OrderItem item : items) {
                ps.setLong(1, order.getId());
                ps.setLong(2, item.getProductId());
                ps.setInt(3, item.getQuantity());
                ps.setBigDecimal(4, item.getUnitPrice());
                ps.addBatch();
            }
            ps.executeBatch();
        }

        String stockSql = "UPDATE products SET stock_qty = stock_qty - ? WHERE id = ? AND stock_qty >= ?";
        try (PreparedStatement ps = conn.prepareStatement(stockSql)) {
            for (OrderItem item : items) {
                ps.setInt(1, item.getQuantity());
                ps.setLong(2, item.getProductId());
                ps.setInt(3, item.getQuantity());
                ps.addBatch();
            }
            int[] results = ps.executeBatch();
        }
    }
}
```

```

        for (int updated : results) {
            if (updated == 0) {
                throw new SQLException("Insufficient stock for one or more items");
            }
        }
    }
}

return order;
}

@Override
public Optional<Order> findById(long id) throws SQLException {
    String sql = "SELECT * FROM orders WHERE id = ?";
    try (Connection conn = dataSource.getConnection();
        PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setLong(1, id);
        try (ResultSet rs = ps.executeQuery()) {
            return rs.next() ? Optional.of(map(rs)) : Optional.empty();
        }
    }
}

@Override
public List<Order> findByBuyer(long buyerId) throws SQLException {
    String sql = "SELECT * FROM orders WHERE buyer_id = ? ORDER BY created_at DESC";
    try (Connection conn = dataSource.getConnection();
        PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setLong(1, buyerId);
        try (ResultSet rs = ps.executeQuery()) {
            List<Order> orders = new ArrayList<>();
            while (rs.next()) {
                orders.add(map(rs));
            }
            return orders;
        }
    }
}

@Override
public List<Order> findBySellerProducts(long sellerId) throws SQLException {
    String sql = "SELECT DISTINCT o.* FROM orders o "
        + "JOIN order_items oi ON oi.order_id = o.id "
        + "JOIN products p ON p.id = oi.product_id "
        + "WHERE p.seller_id = ? ORDER BY o.created_at DESC";
    try (Connection conn = dataSource.getConnection();
        PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setLong(1, sellerId);
        try (ResultSet rs = ps.executeQuery()) {
            List<Order> orders = new ArrayList<>();
            while (rs.next()) {
                orders.add(map(rs));
            }
            return orders;
        }
    }
}

@Override
public List<OrderItem> findItems(long orderId) throws SQLException {
    String sql = "SELECT * FROM order_items WHERE order_id = ?";
    try (Connection conn = dataSource.getConnection();
        PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setLong(1, orderId);
        try (ResultSet rs = ps.executeQuery()) {
            List<OrderItem> items = new ArrayList<>();
            while (rs.next()) {

```

```

        OrderItem item = new OrderItem();
        item.setId(rs.getLong("id"));
        item.setOrderId(rs.getLong("order_id"));
        item.setProductId(rs.getLong("product_id"));
        item.setQuantity(rs.getInt("quantity"));
        item.setUnitPrice(rs.getBigDecimal("unit_price"));
        items.add(item);
    }
    return items;
}
}
}

@Override
public boolean updateStatus(long orderId, String status) throws SQLException {
    String sql = "UPDATE orders SET status = ? WHERE id = ?";
    try (Connection conn = dataSource.getConnection();
        PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setString(1, status);
        ps.setLong(2, orderId);
        return ps.executeUpdate() > 0;
    }
}

private Order map(ResultSet rs) throws SQLException {
    Order order = new Order();
    order.setId(rs.getLong("id"));
    order.setBuyerId(rs.getLong("buyer_id"));
    order.setStatus(Order.Status.valueOf(rs.getString("status")));
    order.setTotalAmount(rs.getBigDecimal("total_amount"));
    Timestamp ts = rs.getTimestamp("created_at");
    if (ts != null) {
        order.setCreatedAt(ts.toLocalDateTime());
    }
    return order;
}
}
}

```